

# Communication Skill for Software and Knowledge Engineers

---

## Lec. #02

# Self-Introduction Presentation

---

# How to explain software library

---

## Why Explaining Software Libraries Well Matters

# Why Explaining Software Libraries Well Matters

---

1. Improves Adoption

2. Helps Onboarding and Collaboration

3. Enables Community Growth

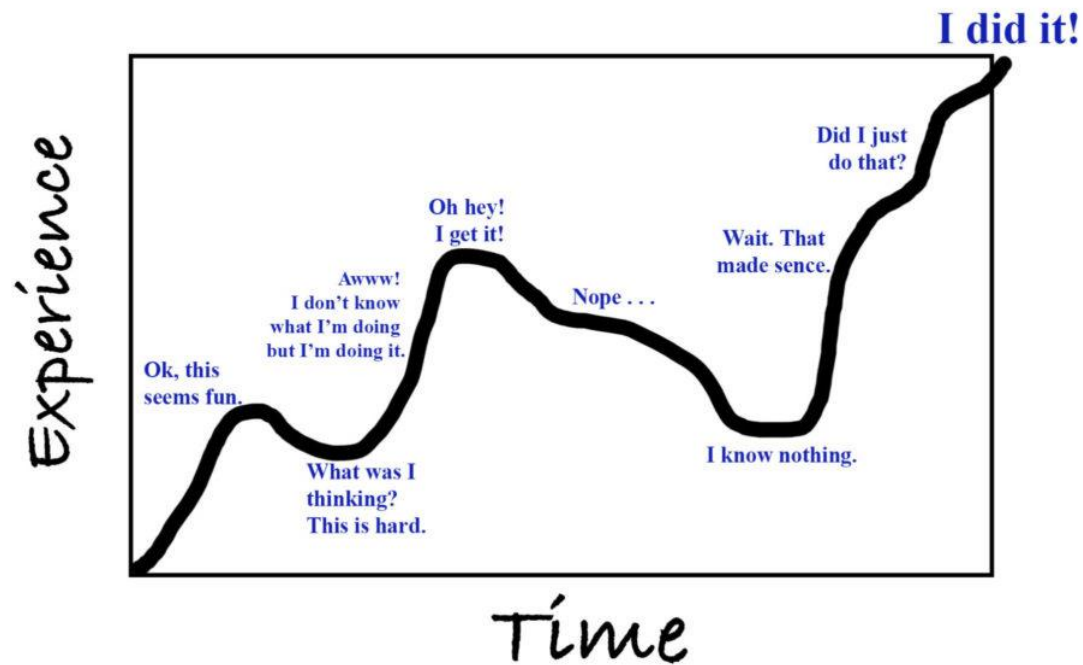
# Why Explaining Software Libraries Well Matters

## 1. Improves Adoption



- Clear explanations reduce the learning curve.

### The Learning Curve



[www.theexcitedwriter.com](http://www.theexcitedwriter.com)

2.

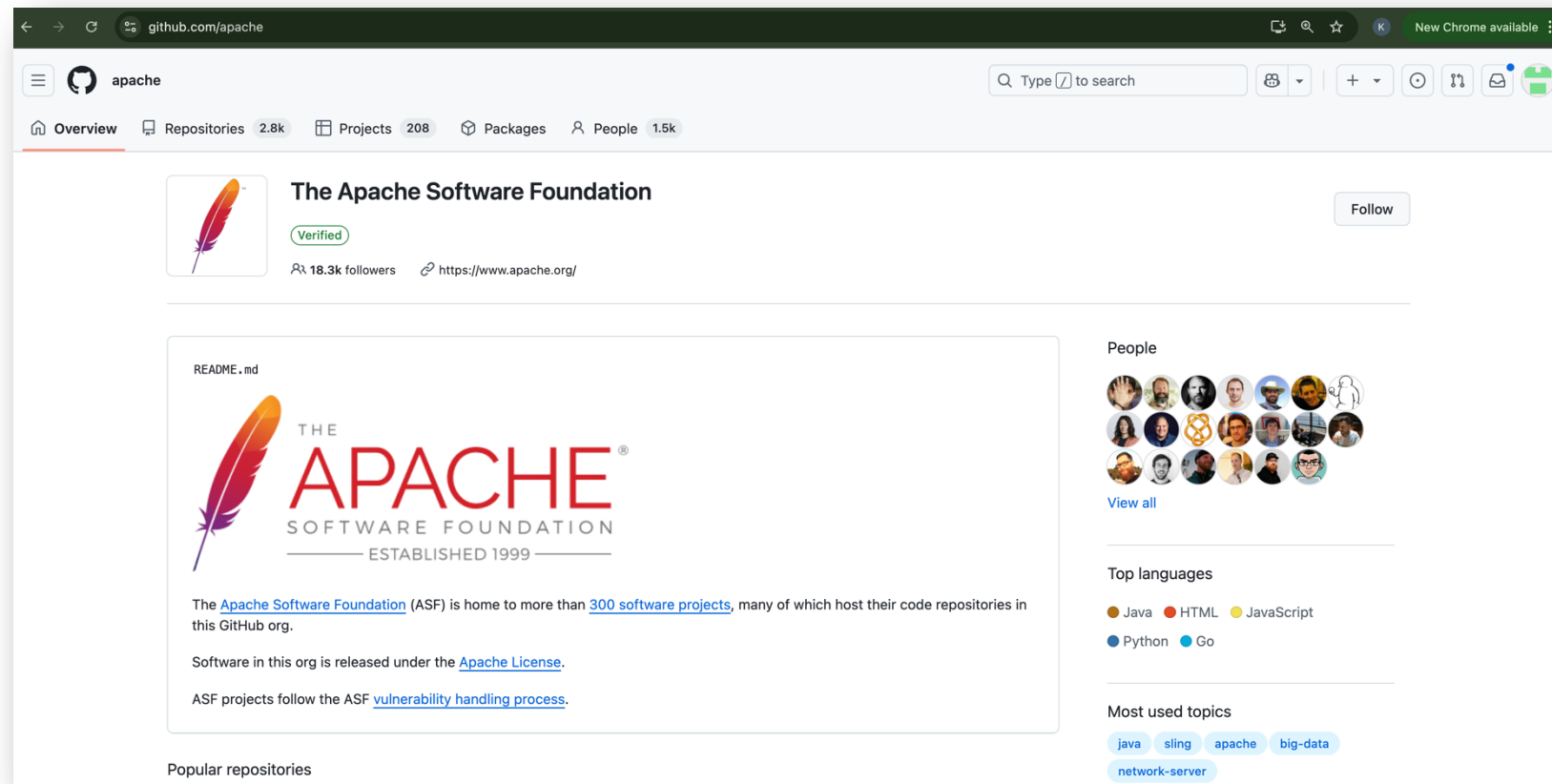
3.

# Why Explaining Software Libraries Well Matters

## 1. Improves Adoption



➤ Developers are more likely to try and trust a well-explained library.



2.

3.

# Why Explaining Software Libraries Well Matters

## 1. Improves Adoption

➤ Great documentation = marketing for developers.



2.  
3.

# Why Explaining Software Libraries Well Matters

## 2. Helps Onboarding and Collaboration



➤ Teams can integrate new tools faster with clear usage examples.

The screenshot shows the pandas documentation homepage. At the top, there's a navigation bar with links: Getting started, User Guide, API reference, Development, and Release notes. A search bar and version selector (2.3 (stable)) are also present. The main heading is "pandas documentation" with a "Show Source" link. Below this, it says "Date: Jun 05, 2025 Version: 2.3.0". There are links for "Download documentation: Zipped HTML" and "Previous versions: Documentation of previous pandas versions is available at pandas.pydata.org.". A section of "Useful links" includes Binary Installers, Source Repository, Issues & Ideas, Q&A Support, and Mailing List. A paragraph describes pandas as an open source, BSD-licensed library providing high-performance, easy-to-use data structures and data analysis tools for the Python programming language. Two main cards are visible: "Getting started" with a running person icon and "User guide" with an open book icon. Both cards provide brief descriptions and buttons to navigate to the respective guides.

The screenshot shows a specific page from the pandas documentation titled "How to calculate summary statistics". The page has a sidebar with a search bar and version selector (2.3 (stable)). The main content area is titled "Titanic data" and shows a code snippet for loading and displaying the Titanic dataset. Below the code, there's a table showing the first 5 rows of the dataset. The table has columns: PassengerId, Survived, Pclass, Fare, Cabin, and Embarked. The data shows the first 5 passengers, with their survival status, class, fare, cabin number, and embarkation point. Below the table, there's a section titled "Aggregating statistics" with a diagram showing a grid of data being processed into a single summary value. The diagram shows a 5x3 grid of squares, with the first column highlighted in yellow. An arrow points from this grid to a single pink square, representing the aggregation process. Below the diagram, there's a question: "What is the average age of the Titanic passengers?". The code snippet shows the calculation: `In [4]: titanic["Age"].mean()` and the output: `Out[4]: 29.69911764705882`. The page also has a "Show Source" link.

3.

# Why Explaining Software Libraries Well Matters

## 2. Helps Onboarding and Collaboration



➤ Reduces time spent answering repeated questions.

The screenshot shows the pandas User Guide website. The main heading is "Frequently Asked Questions (FAQ)". Below it, the section "DataFrame memory usage" is highlighted. The text explains that the memory usage of a DataFrame (including the index) is shown when calling the `info()` method. A configuration option, `display.memory_usage`, specifies if the DataFrame memory usage will be displayed when invoking the `info()` method. For example, the memory usage of the DataFrame below is shown when calling `info()`:

```
In [1]: dtypes = [
...:     "int64",
...:     "float64",
...:     "datetime64[ns]",
...:     "timedelta64[ns]",
...:     "complex128",
...:     "object",
...:     "bool",
...: ]
...:

In [2]: n = 5000

In [3]: data = {t: np.random.randint(100, size=n).astype(t) for t in dtypes}

In [4]: df = pd.DataFrame(data)

In [5]: df["categorical"] = df["object"].astype("category")

In [6]: df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 8 columns):
#   Column              Non-Null Count  Dtype
#   ...
```

On the right side, there is a list of topics covered on the page, including "DataFrame memory usage", "Using if/truth statements with pandas", "Mutating with User Defined Function (UDF) methods", "Missing value representation for NumPy types", "Differences with NumPy", "Thread-safety", and "Byte-ordering issues". A "Show Source" link is also present.

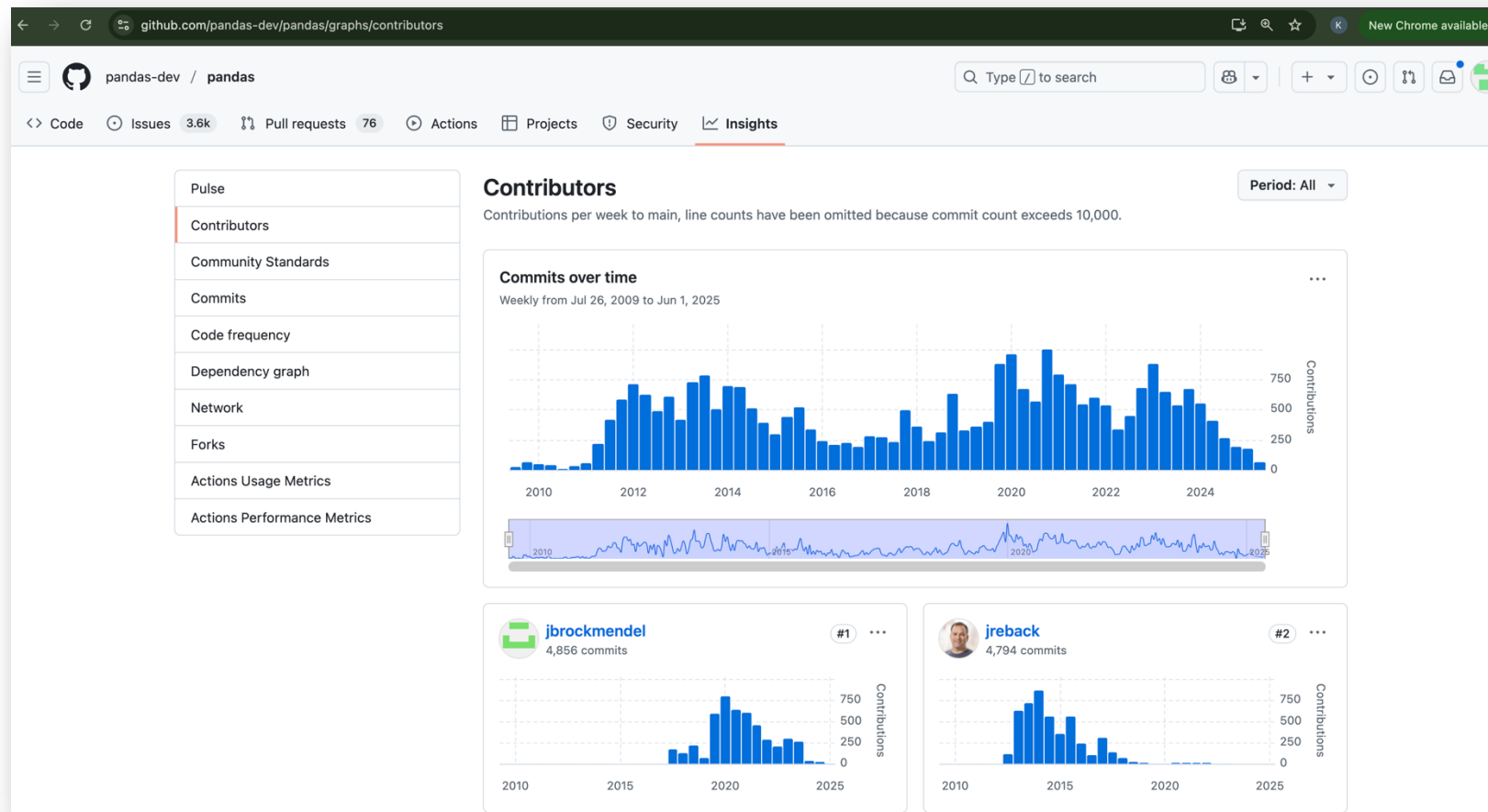
3.

# Why Explaining Software Libraries Well Matters

## 2. Helps Onboarding and Collaboration



- New contributors or developers ramp up more easily.



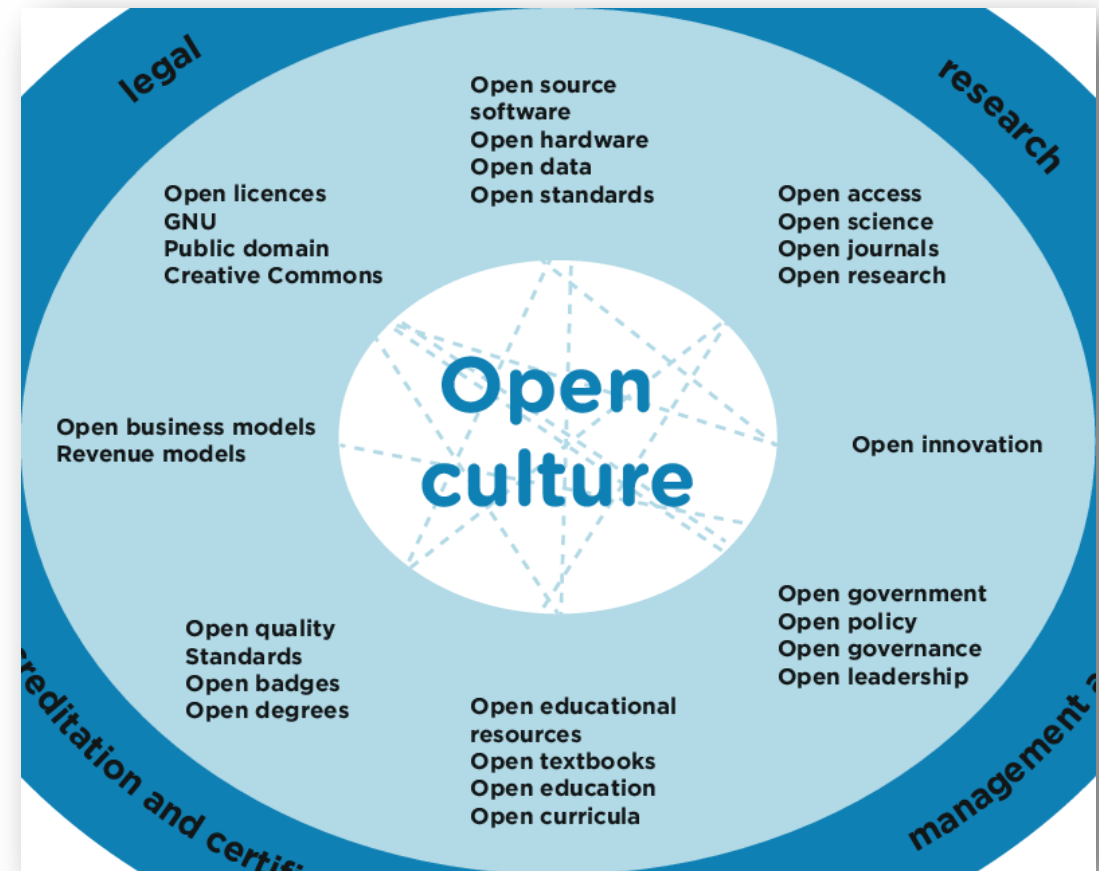
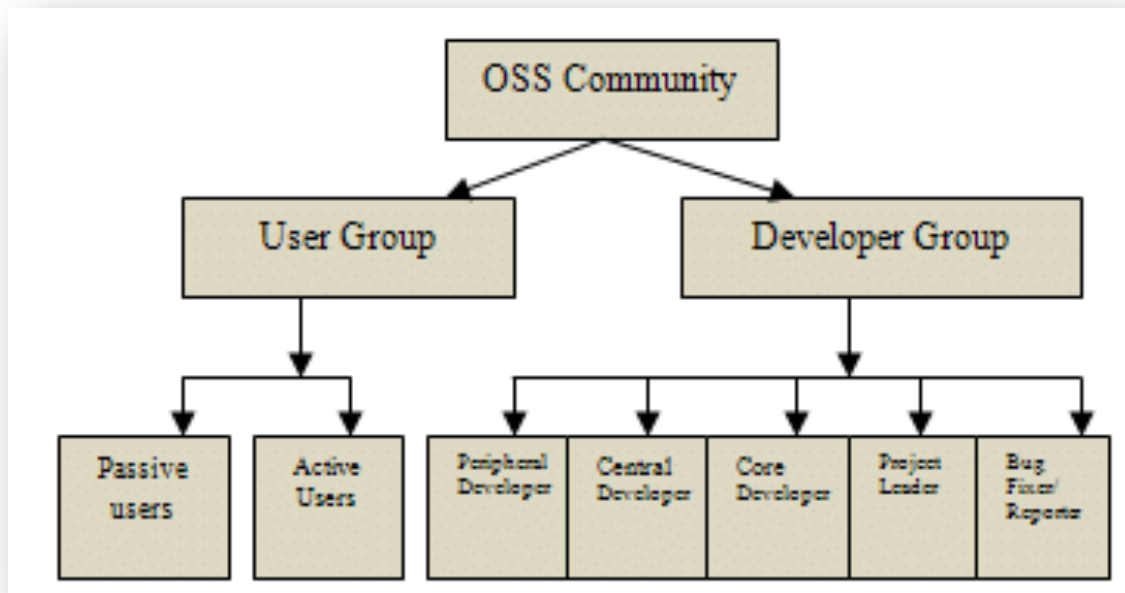
3.

# Why Explaining Software Libraries Well Matters

## 3. Enables Community Growth



- Good explanations attract contributors and open-source supporters.

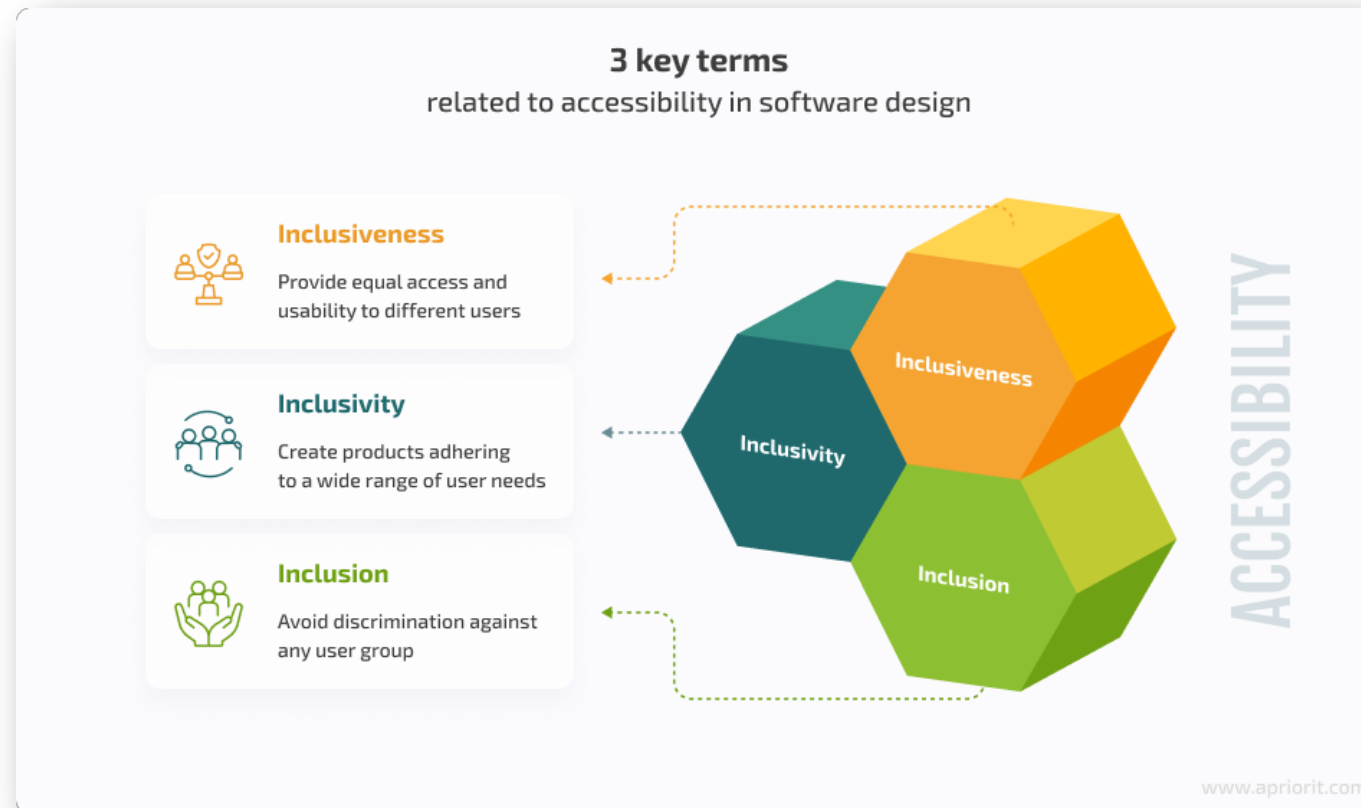


# Why Explaining Software Libraries Well Matters

## 3. Enables Community Growth



- Makes the project more inclusive and accessible.



# Why Explaining Software Libraries Well Matters

## 3. Enables Community Growth



- Encourages ecosystem development (plugins, wrappers, integrations).



# Why Explaining Software Libraries Well Matters

---



# Purpose and Design Goals

---

## Why does this library exist?

- What problem does it solve?
- What are its main use cases?
- How does it compare to alternatives?
- What is the design philosophy?
  - simplicity, performance, flexibility

## Numpy

- ✓ It is designed for efficient numerical computations with arrays
- ✓ It replaces slower Python loops with vectorized operations.

# Core Components

---

## What are its building blocks?

### ➤ **Functions**

- What are the most-used functions?

### ➤ **Classes**

- Are there key abstractions (e.g., Model, Request, Layer)?

### ➤ **Modules**

- How is the code organized? Are there submodules?

### ➤ **Patterns**

- Are there common design patterns or idioms users must follow?

# Core Components

---



## Requests

- **Core function:**
  - `requests.get()`, `requests.post()`
- **Objects:**
  - `Response`, `Session`
- **Pattern:**
  - Use a session for persistent headers/auth

# Core Components

---

## Know what it needs to work.

- Does it rely on any other libraries (e.g., NumPy, Flask)?
- Are there OS, Python version, or hardware requirements?
- Is it compatible with Jupyter notebooks, web frameworks, etc.?

## Matplotlib

It depends on NumPy and works best in a Jupyter environment for visual outputs.

NumPy 

# Output and Expected Behavior

---

- What does the library return, generate, or produce?
- What types of output does it give (objects, text, plots, logs)?
- Is the output predictable and consistent?
- Are there side effects (e.g., writes to file, network calls)?

# Output and Expected Behavior

---

**scikit-learn:**



- Fit methods return trained model objects
- Predict returns NumPy arrays.

# Versioning and Stability

---

**How stable is it?**

**Is it actively maintained?**

- Semantic versioning? (v1.x.x = stable, v0.x = evolving)
- Is it production-ready or experimental?
- How often does it change APIs or deprecate features?
- Where can you check changelogs or roadmaps?

# Versioning and Stability

---

## TensorFlow



Major changes between v1.x and v2.x  
, code written for v1 might break in v2

# Versioning and Stability



github.com/tensorflow/tensorflow

|                    |                                                                 |                |
|--------------------|-----------------------------------------------------------------|----------------|
| tensorflow         | remove old copy of tf_quantize_op and rename new one            | 1 hour ago     |
| third_party        | [XLA:GPU] Add triton support test for convolution op            | 42 minutes ago |
| tools              | Added comments                                                  | 3 months ago   |
| .bazelignore       | Migrate TensorFlow on newest Hermetic Python set of rul...      | last year      |
| .bazelrc           | Undo unintentional edits to oneDNN configs from cl/7559...      | 2 weeks ago    |
| .bazelversion      | Upgrade to Bazel 7.4.1 for TensorFlow                           | 4 months ago   |
| .clang-format      | [.clang-format] Init @ root                                     | 4 years ago    |
| .gitignore         | Added venv/ to .gitignore                                       | 7 months ago   |
| .pylintrc          | Add soft-link to pylintrc to project root                       | 6 years ago    |
| .zenodo.json       | Add .zenodo.json for clean automated DOI numbers.               | 4 years ago    |
| AUTHORS            | Add Arm Ltd to AUTHORS                                          | 3 years ago    |
| BUILD              | Refactor mechanisms of building TF wheel and storing TF...      | 3 months ago   |
| CITATION.cff       | Add CITATION.cff                                                | 4 years ago    |
| CODEOWNERS         | Remove myself from CODEOWNERS as I will no longer be ...        | 3 years ago    |
| CODE_OF_CONDUCT.md | Update CODE_OF_CONDUCT and fix broken sync                      | 4 years ago    |
| CONTRIBUTING.md    | Adjust the build config to an existing value defined in .baz... | 5 months ago   |
| ISSUES.md          | Internal change                                                 | 4 years ago    |
| LICENSE            | Remove duplicate LICENSE information for third_party de...      | 4 years ago    |
| README.md          | Remove outdated build badges                                    | 2 months ago   |

deep-neural-networks deep-learning  
neural-network tensorflow ml  
distributed

Readme  
Apache-2.0 license  
Code of conduct  
Security policy  
Cite this repository  
Activity  
Custom properties

190k stars  
7.5k watching  
74.7k forks

Report repository

Releases 216

TensorFlow 2.19.0 (Latest)  
on Mar 12

+ 215 releases

Packages

No packages published

Used by 525k

+ 525,091

# Versioning and Stability



tensorflow / tensorflow

Q Type [Z] to search

887

462

2

427

Insights

Releases

Tags

Find a release

Mar 12

tensorflow-jenkins

v2.18.1

cb64295

Compare

## TensorFlow 2.18.1

### Release 2.18.1

#### Security

- Updates curl to 8.11.0 to handle [CVE-2024-2004](#), [CVE-2024-2379](#), [CVE-2024-2398](#), [CVE-2024-2466](#), [CVE-2024-6197](#), [CVE-2024-7264](#), [CVE-2024-8096](#) and [CVE-2024-9681](#).

#### Bug Fixes and Other Changes

- Loosen `ml_dtypes` upperbound to `< 1.0.0` to reduce conflicts when installed with other ML ecosystem components.

#### Breaking Changes

- `tf.lite`
  - Interpreter:
    - `tf.lite.Interpreter` gives warning of future deletion and a redirection notice to its new location at `ai_edge_litert.interpreter`. See the [migration guide](#) for details.
- Tensorflow-tpu for this patch is skipped due to some sparsecore related bugs. We suggest to upgrade to 2.19.0 instead.

# Identify the Audience

---

## Why Audience Matters

- ✓ A great explanation is not **one-size-fits-all** it should match the background, goals, and expectations of the listener.
- ✓ Speaking to the wrong level leads to confusion or boredom.
- ✓ You should **adapt the depth, terminology, and examples** to fit the audience.

Identify the Audience

# Audience by **Experience Level**

---

 **Beginner**

 **Intermediate**

 **Advanced**

# Audience by **Experience Level**

---

## **Beginner**

“What is it? Why should I care?”

- ✓ Needs motivation and big-picture understanding.
- ✓ Avoid deep internals or low-level syntax.
- ✓ Use analogies, diagrams, and minimal code.



# Audience by **Experience Level**

---

## **Beginner**

“What is it? Why should I care?”

"Requests lets you send data to websites and get responses — like using a browser, but in your Python code."



# Audience by **Experience Level**

---

## **Intermediate**

“How do I use this effectively?”

- ✓ Already understands the basic concept.
- ✓ Interested in API patterns, best practices, and real-world examples.
- ✓ Wants to know common mistakes and performance tips.



# Audience by **Experience Level**

---

## **Intermediate**

“How do I use this effectively?”

"To keep session data like cookies across requests, use a Session object — it's more efficient than calling `get()` repeatedly."



# Audience by **Experience Level**

---

## **Advanced**

“How does it work under the hood?”

- ✓ Curious about architecture, extensibility, and edge cases.
- ✓ May want to customize or contribute to the library.
- ✓ Can handle abstract or technical language.

# Audience by **Experience Level**

---

## **Advanced**

“How does it work under the hood?”

"Under the hood, requests wraps urllib3, and handles connection pooling, SSL verification, and redirects using a layered adapter system."

Identify the Audience

# Audience by Role or Domain

---



Developers



Data Scientists / ML Engineers



Product / Business Teams

# Audience by Role or Domain

---

## Developers

- Focus on integration, performance, and error handling.
- Interested in API stability and documentation.



# Audience by Role or Domain

---



## Data Scientists / ML Engineers

- Care about usability in notebooks, compatibility with data formats.
- Prefer visual examples and reproducible code.



Identify the Audience

# Audience by Role or Domain

---



## Product / Business Teams

- Want high-level benefits, not code.
- Emphasize outcomes: speed, UX, cost savings, etc.

# Identify the Audience

| Audience            | Focus                      | Method                        | Output Examples          |
|---------------------|----------------------------|-------------------------------|--------------------------|
| <b>Beginner</b>     | Concept & motivation       | Diagrams, analogies, 1-liners | "This makes HTTP easy"   |
| <b>Intermediate</b> | Practical usage & patterns | Code snippets, examples       | Working scripts          |
| <b>Advanced</b>     | Internals, customization   | Source code, design diagrams  | Custom components/ hooks |

# Structure the Explanation

---

## The 4-Layer Approach

A clear explanation follows a **natural learning progression** from *why* to *what* to *how* to *what if*.

Layer 1: **Motivation / Problem**

Layer 2: **Core Concept or Abstraction**

Layer 3: **API Examples / Usage Patterns**

Layer 4: **Edge Cases / Gotchas**

# Structure the Explanation

---

## Layer 1: **Motivation / Problem**

“Why should I care? What pain does this solve?”

- Start with a real-world problem or scenario.
- Keep it relatable and grounded in user needs.
- Optionally: compare the “before” and “after” using the library.

Layer 2:

Layer 3:

Layer 4:

# Structure the Explanation

---

## Layer 1: **Motivation / Problem**

“Why should I care? What pain does this solve?”

"Imagine you're analyzing CSV files full of messy sales data with raw Python. It's slow and verbose. Pandas solves this by giving you a fast, spreadsheet-like data structure in code."

Layer 2:

Layer 3:

Layer 4:

# Structure the Explanation

---

## Layer 2: **Core Concept or Abstraction**

“What is the main idea? What’s the mental model?”

- Define key abstractions: e.g., DataFrame, Request, Layer, Component.
- Use analogies or diagrams to help intuition.
- Don’t just describe *what* it does — explain *how to think about it*.

Layer 3:

Layer 4:

# Structure the Explanation

---

## Layer 2: **Core Concept or Abstraction**

“What is the main idea? What’s the mental model?”

”     *A Flask app is like a restaurant. Routes are like the menu  
— each URL corresponds to a function that serves a response.*”

Layer 3:

Layer 4:

# Structure the Explanation

---

## Layer 3: **API Examples / Usage Patterns**

“How do I actually use this in code?”

- Show short, focused code snippets (not full apps yet).
- Focus on common workflows, not edge features.
- Add inline comments or walk through each line verbally.

Layer 4:

# Structure the Explanation

---

## Layer 3: **API Examples / Usage Patterns**

“How do I actually use this in code?”

```
import requests
response = requests.get("https://api.example.com/data")
print(response.status_code)
print(response.json())
```

Layer 4:

# Structure the Explanation

---

## Layer 4: **Edge Cases / Gotchas**

“What can go wrong? What should I look out for?”

- Highlight common beginner mistakes or silent failures.
- Mention performance traps, error-prone areas, version quirks.
- Provide clear debugging tips or safe defaults.

# Structure the Explanation

---

## Layer 4: **Edge Cases / Gotchas**

“What can go wrong? What should I look out for?”

"Be careful with chained assignments like `df[df.col > 0]['col'] = 1` — it might not actually modify the DataFrame. Use `.loc[]` instead."

# Structure the Explanation

## Techniques to Enhance Clarity

---

### Use Visuals

- ❖ **Architecture diagrams** (for internal flow)
- ❖ **Flowcharts** (for user interaction or data flow)
- ❖ **Before/after comparisons** (code without vs. with library)

# Structure the Explanation

## Techniques to Enhance Clarity

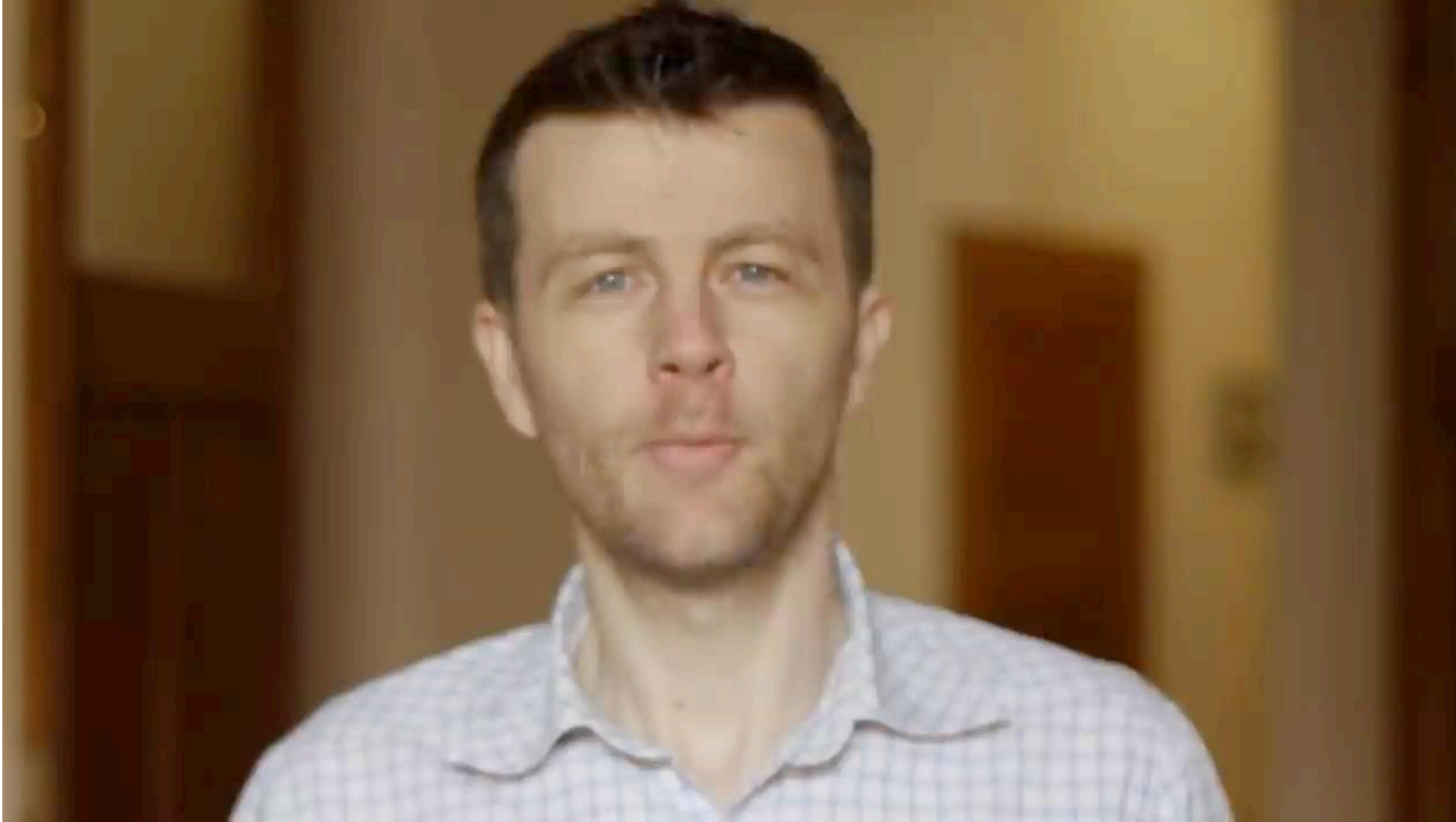
---

### **Use Analogies & Metaphors**

- ❖ Flask app
  - restaurant with routes as dishes
- ❖ NumPy array
  - Excel grid with superpowers
- ❖ Transformer model
  - attention as highlighters on a document

# Pandas

---



← → ↻ pypi.org ☆ ⬇️ K New Chrome available

2025 Python Packaging Survey is now live! [Take the survey now](#) ✕

 Help Docs Sponsors Log in Register

# Find, install and publish Python packages with the Python Package Index

Search projects 🔍

Or [browse projects](#)

643,812 projects 7,022,964 releases 14,475,551 files 931,376 users



**python**  
Package Index<sup>TM</sup>

The Python Package Index (PyPI) is a repository of software for the Python programming language.

PyPI helps you find and install software developed and shared by the Python community. [Learn about installing packages](#).

Package authors use PyPI to distribute their software. [Learn how to package your Python code for PyPI](#).

# Q&A

---

Email: [kundjanasith.th@ku.th](mailto:kundjanasith.th@ku.th)